第一轮完整测试结果汇总（按 `test_metric_for_4RQs.txt` 口径）

> 说明：
>
> 1. **表组 A**：与 PMD / SpotBugs 的比较
> 2. **表组 B**：模型比较（效果 + 成本），同时回答 RQ2 与 RQ3
> 3. **表组 C**：模块消融实验
>
> 其中：
>
> - **Juliet**：
> - PMD / SpotBugs 使用严格的 **case-level**（把一个测试样例文件中的 bad 方法看作一个整体缺陷单位）口径（bad case / good case）
> - JLeakDetector（当前方法）自身更适合用 bug-unit（每一个具体的关键资源对象，都是一个独立缺陷单位）口径评估，但在与 PMD / SpotBugs 横向比较时，这里统一给出 case-level 指标
> - **真实项目**：
> - 按 `test_metric_for_4RQs.txt` 的要求，不再使用 FP / TN / F1 作为主指标
> - 主要关注 `TP_known` / `FN` / `recall_known` / `issue_count` / `inspection_burden` / `时间` / `token`

表组 A：与 PMD / SpotBugs 的比较

表 A1：Juliet 上 PMD、SpotBugs 与 JLeakDetector最好/最坏结果的严格比较

方法	TP	FP	FN	TN	Precision	Recall	F1
PMD	8	8	29	88	0.500	0.216	0.302
SpotBugs	6	0	31	96	1.000	0.162	0.279
JLeakDetector-best	36	1	1	95	0.973	0.973	0.973
JLeakDetector-worst	36	20	1	76	0.643	0.973	0.774

对应结论

- 即使在最差运行结果下，JLeakDetector的 F1 仍明显高于 PMD 和 SpotBugs。
- PMD 的主要问题是 **FP 较多**，容易把 good case 也报出来。
- SpotBugs 的主要问题是 **Recall 偏低**，虽然 Precision 很高，但覆盖能力不足。
- JLeakDetector的波动主要体现为 **FP 数量变化**，而不是召回崩塌。

表 A2：FitNesse 上 PMD、SpotBugs 与 JLeakDetector 最好/最坏结果的比较

方法	TP_known	FN	recall_known	候选结果数	inspection_burden
PMD	3	2	0.6000	51.00	10.20
SpotBugs	5	0	1.0000	10.00	2.00
JLeakDetector-best	5	0	1.0000	67.33	12.47
JLeakDetector-worst	2	3	0.4000	60.00	11.60

对应结论

- 在 FitNesse 上，SpotBugs 与 JLeakDetector 最佳配置都能达到 **1.0 recall_known**。
- 但 JLeakDetector 在该项目上的波动较大，最差情况下会下降到 **0.4**，说明其对流程配置非常敏感。
- SpotBugs 在 FitNesse 上表现异常突出，而且人工审核负担最低。
- PMD 在 FitNesse 上覆盖能力偏弱，仅命中 3 / 5 个已知缺陷。

表 A3：TripleA 上 PMD、SpotBugs 与 JLeakDetector 最好/最坏结果的比较

方法	TP_known	FN	recall_known	候选结果数	inspection_burden
PMD	12	2	0.8571	124.00	8.86
SpotBugs	7	7	0.5000	26.00	1.86
JLeakDetector-best	14	0	1.0000	90.00	5.14
JLeakDetector-worst	12	2	0.8571	88.33	5.21

对应结论

- 在 TripleA 上，JLeakDetector 最佳结果明显优于 PMD 和 SpotBugs。
- JLeakDetector worst退化后大约与 PMD 持平，但仍显著优于 SpotBugs。
- PMD 覆盖较高，但候选报告数过多，人工审核负担较重。
- SpotBugs 候选较少，但已知缺陷覆盖明显不足。

表组 B：模型比较（效果 + 成本）

表 B1：Juliet 上 PMD、SpotBugs、qwen-plus、deepseek-chat、doubao 的比较

方法	TP	FP	FN	Precision	Recall	F1
PMD	8.00	8.00	29.00	0.500	0.216	0.302

方法	TP	FP	FN	Precision	Recall	F1
SpotBugs	6.00	0.00	31.00	1.000	0.162	0.279
deepseek-chat	35.00	3.67	2.00	0.906	0.946	0.925
qwen-plus	34.75	3.92	2.25	0.901	0.939	0.919
doubao-seed-2-0-mini-260215	35.25	11.83	1.75	0.754	0.953	0.840

对应结论

- 三个 JLeakDetector 模型都显著优于 PMD 和 SpotBugs。
- `deepseek-chat` 与 `qwen-plus` 的综合效果最均衡。
- `doubao` 的 Recall 很高，但 FP 明显偏多，因此 F1 不如 deepseek / qwen。
- 这说明在 Juliet 上，JLeakDetector 的核心优势在于 **高召回且保持较高 Precision。**

表 B2：FitNesse 上 PMD、SpotBugs、qwen-plus、deepseek-chat、doubao 的比较

方法	TP_known	FN	recall_known	候选结果数	inspection_burden	平均时间（秒）	平均 token
PMD	3.0	2.0	0.6000	51.00	10.20	N/A	N/A
SpotBugs	5.0	0.0	1.0000	10.00	2.00	N/A	N/A
deepseek-chat	3.5	1.5	0.7000	64.25	12.15	838.54	381,632.8
qwen-plus	3.5	1.5	0.7000	68.25	12.95	465.42	349,796.6
doubao-seed-2-0-mini-260215	3.5	1.5	0.7000	65.33	12.37	1760.16	1,179,680.2

对应结论

- 在 FitNesse 上，SpotBugs 是最强的传统基线工具，直接达到 **1.0 recall_known。**
- JLeakDetector 三个模型的平均值相同，说明该项目主要受**配置影响**，而不是模型差异影响。
- 成本方面：
  - `qwen-plus` 最快，token 成本也最低；
  - `doubao` 最慢，且 token 成本最高。

表 B3：TripleA 上 PMD、SpotBugs、qwen-plus、deepseek-chat、doubao 的比较

方法	TP_known	FN	recall_known	候选结果数	inspection_burden	平均时间（秒）	平均 token
PMD	12.0	2.0	0.8571	124.00	8.86	N/A	N/A
SpotBugs	7.0	7.0	0.5000	26.00	1.86	N/A	N/A
deepseek-chat	13.0	1.0	0.9286	88.92	5.17	921.27	637,590.2
qwen-plus	13.0	1.0	0.9286	98.42	5.83	853.37	765,097.2
doubao-seed-2-0-mini-260215	13.0	1.0	0.9286	94.42	5.56	1550.02	1,845,934.2

对应结论

- 在 TripleA 上，三种 JLeakDetector 模型的平均 recall_known 都高于 PMD，也显著高于 SpotBugs。
- PMD 的覆盖较高，但候选结果数太多，人工审核负担较重。
- SpotBugs 候选较少，但 known-bug 覆盖明显不足。
- 三个模型里：
  - `qwen-plus` 最快；
  - `doubao` 最贵；
  - `deepseek-chat` 在效果与成本之间更均衡。

表组 C：模块消融实验

> 说明：当前三个模型在消融趋势上高度一致，因此这里使用 `deepseek-chat` 作为代表进行展示。
>
> issue-first模块：一个前置 source 聚类与代表选择模块。它先将多个可能属于同一问题的 source 按 component 聚类，再从每个 component 中选取少量代表性 source（witness）进入后续完整分析流程，而不是对所有 source 全量展开。其主要作用是缓解 source 爆炸、降低候选路径规模、降低整体分析成本以及使检测流程更接近 issue 级审计视角（把语义上相关的多条证据合并成一个实际需要人工检查的问题）。
>
> soot模块：利用 Soot 生成的静态语义事实，对候选资源泄露路径进行前置检查，在进入 LLM 路径验证之前剪除一部分明显不成立或证据不足的路径。其作用为减少需要送入 LLM 的候选路径数量、降低时间与 token 成本以及在部分项目上抑制噪声报告。

表 C1：FitNesse 上 JLeakDetector 消融结果

变体	recall_known	issue_count	inspection_burden	平均时间（秒）	平均 token
baseline	0.4000	60.00	11.60	898.88	350,484.3

变体	recall_known	issue_count	inspection_burden	平均时间(秒)	平均token
no_issue_first	0.4000	60.67	11.73	763.44	403,441.3
no_soot	1.0000	67.67	12.53	772.94	374,067.3
no_soot_no_issue_first	1.0000	68.67	12.73	918.91	398,538.3

对应结论

- FitNesse 上最关键的影响因素是 **soot**。
- 关闭 soot 后，recall_known 从 0.4 直接提升到 1.0，但时间和token成本均有上升。
- `issue-first` 单独关闭并不能带来实质改善。
- 因此可以认为：**当前 soot 预过滤在 FitNesse 上存在明显负效应，但对于时间和token成本的压缩上可以起到较为明显的作用**

表 C2：TripleA 上 JLeakDetector 消融结果

变体	recall_known	issue_count	inspection_burden	平均时间(秒)	平均token
baseline	0.8571	88.33	5.21	1019.63	604,449.3
no_issue_first	0.9286	86.00	5.00	841.84	673,212.7
no_soot	0.9286	91.33	5.31	971.97	601,459.3
no_soot_no_issue_first	1.0000	90.00	5.14	851.64	671,239.7

对应结论

- TripleA 上 soot 和 issue-first 都会带来一定损失。
- 同时关闭二者时效果最好。
- 与 FitNesse 不同，TripleA 上并不存在 soot 的单因素压倒性负效应，而是 soot 与 issue-first 都需要重新评估。
- 这说明当前系统的核心瓶颈之一在于：**模块配置策略与项目特性之间并不匹配。**

总体总结

1. 与传统工具相比

- 在 **Juliet** 上，JLeakDetector 无论最好还是最坏结果，都显著优于 PMD 与 SpotBugs。
- 在 **真实项目** 上：
  - FitNesse：SpotBugs 最强，JLeakDetector 最佳配置可追平；
  - TripleA：JLeakDetector 明显优于 PMD 和 SpotBugs。
- 总体而言，JLeakDetector的表现略优于FitNesse和TripleA。

2. 模型比较

- `qwen-plus` 是当前**综合性价比最优模型**；
- `deepseek-chat` 在效果和成本之间较均衡；
- `doubao` 的成本显著偏高，且 Juliet 上 FP 较多，不适合作为当前主力模型。

3. 模块作用

- 当前模块配置（ soot 和 issue-first）对结果的影响大于模型差异本身；
- FitNesse 上 soot 明显有负效应；
- TripleA 上 soot 和 issue-first 同时关闭效果最好。
- 当前系统的核心瓶颈之一在于**模块配置策略与项目特性之间的匹配性需要人为探索。**

4. 系统定位

- 当前系统已经表现出较强的 benchmark 覆盖能力和真实项目已知 bug 发现能力；
- 但 `inspection_burden` 仍然偏高，说明它更适合作为**高召回、辅助式的资源泄露审计工具**，而非低噪声、生产级自动告警工具。考虑到由于时间限制无法对所有的候选进行人工确认，就目前的效果表现来看，在某些真实项目（inspection_burden值在5左右）上可以展现较好的实用价值。